

[C] “Yeah, Right...” Sarcasm Classification on Social Media

Arvid Kolstad
(Dated: June 4, 2026)

Social behavior in humans are a complex subject that can seem almost impossible to fully understand. A good example of this are usage of sarcasm in interactions, which is famously easily to create misunderstandings, especially when written, because of intonation and voice features are often used to clear markers for a sarcastic comment. Combined with a internet where endless of conversations take place everyday about the most obscure topics makes this a complicated and nuanced classification problem. This creates the interesting question: How well can the the machines, which we run these websites on, understand what is really being said? Therefore this project aims to explore how well different machine learning models can classify sarcasm from sentences made on social media.

Project Topic: Sarcasm Detection in Social Media

Teaching Assistant: Antonio Ciarlo

I. INTRODUCTION

Human communication is a rich and multifaceted phenomenon, shaped by tone, context, cultural background, and shared understanding between speakers. Among its many complexities, sarcasm stands out as a particularly challenging form of expression, one that relies heavily on non-verbal cues such as intonation, facial expression, and vocal emphasis to convey a meaning that is often the direct opposite of what is literally stated. While this poses little difficulty in face-to-face conversation, the written word strips away these contextual signals, leaving the reader to infer intent from text alone.

This challenge is amplified dramatically in the context of social media, where vast volumes of informal, unstructured text are produced daily across platforms dedicated to virtually every topic imaginable. On these platforms, sarcasm is not only common but often central to the culture of communication, used for humor, criticism, and commentary alike. The absence of prosodic cues in this medium means that sarcastic remarks are frequently misread, even by human readers familiar with the context.

For automated systems tasked with making sense of this content, whether for sentiment analysis, content moderation, or opinion mining, the accurate detection of sarcasm represents a significant open problem. A model that fails to recognize sarcasm will systematically misinterpret the sentiment and intent behind a large portion of online discourse, undermining the reliability of any downstream analysis.

This report investigates the capacity of machine learning models to address this problem by classifying sarcastic statements drawn from social media text. By evaluating and comparing different modeling approaches, this work aims to shed light on how well computational systems can grasp one of the more elusive dimensions of human language.

II. OVERVIEW

In this section a brief summary of the models used in this report can be found. To every model, there is a short discussion about it's advantages and weak point and an argument why the model was used.

Method 1 - Logistic Regression.

This model is rather simple and uses only a linear layer to classify the sentence from the vectorization. For this it uses the pre-trained model FastText [1] which is a more complex version of the famous Word2Vector. It can handle the problem of Out-Of-Vocabulary words by breaking up longer words into smaller parts that it does understand. This is great for social media text because of it's higher frequency of typos and slang. Due to the models simplicity, it does not understand words connection to other words and therefore this is a good model to use as a benchmark.

Method 2 BiLSTM with multi-head attention.

This model uses a recurrent network that takes each word vector, both from the start of the sentence, and the end of the sentence, and feeds these trough with the next word as input. After this the hidden states of the network gets fed into an attention network which learns how different words connects other. This is a good in between the Logistic Regression and the BERT-model and will hopefully display the importance of connections between words when it comes to classifying sarcasm.

Method 3 BERT.

This model leans heavily on the pre-trained BERTweet model [2], and leverages it's understanding of language to try to predict sarcasm. The BERTweet is a transformer encoder that first has been trained on a big dataset of english, and then have been fine tuned on 850 million tweets. This makes it good at understanding internet language and slang, which makes it perfect for this classification problem. It's without question the biggest model in this project and has over 100 million parameters, and has the best chances of scoring the highest.

TABLE I. **Overview of ML models used in this project.** Table where the different models used in the project is presented, their features explained and arguments for their fit to the project.

Method	Use case scenario	Features	Suitable for the project?
Logistic regression	Classification Model	Fast and simple model that is easy to interpret and train. Looks at the importance of each word in the sentence to classify it. Probably too simple for classifying harder sentences.	Well studied method that is used for getting a base line performance on the data set.
BiLSTM with multi-head attention	Classification Model	Can capture word order and connect words over larger distances in sentences which is important for sarcasm. The attention mechanism can learn what words refers to each other and therefore understand the sentence better.	Medium size model that had great performance for its time.
BERTweet	Classification Model	An already pre-trained language model that can be used to classify sarcasm by fine tuning it by setting a linear layer after it and finetuning the whole model on the specific data set. The model is quite large and therefore it can be computationally intensive to train.	A large model that has a good understanding of our language, which makes it perfect for using in this classification problem.

III. METHOD

B. FastText

In this section we will first introduce the dataset and then we will go through the different models architecture and then explain the training pipeline. All the models are built in python by using the library PyTorch.

A. Dataset

The dataset that has been used in this project is the following [3]. It's a dataset that was retrieved from the website Kaggle.com, and has been created by scraping the the social platform Reddit.com for comments. On the website you can put different labels on your comments to signal what the comment is about. One of these labels stands for that the comment has some sarcastic remarks in it, and this is what is used for classifying the dataset. The total number of comments is exactly 1 million and the dataset is perfectly balanced so 500K is labeled as normal and the other half is labeled as sarcastic. The dataset also contained the text that the comment answered to, what the authors name was, the name of the specific chat forum that the comment was written in and much more but the only data that was used in this project was the original comment and it's sarcasm label.

For both this model and the BiLSTM, the word embedder FastText is used [1]. It is a pre trained model that takes a word and transforms it into a 300 dimensional vector, and therefore makes it easier interpretable to a computer. This model was picked for the project for it's ability to handle Out-Of-Vocabulary words. If it stumbles upon a word that it doesn't recognize it tries to break down the word in smaller words that could make the word make sense. This is great for handling slang and spelling error that might not be included in the FastText set of known words. This model only handles word so special characters are removed before converting the word to a sentence. For sarcasm in text, these provide valuable markers so to not loose this information, 4 sarcasm pointer where extracted from each sentence. The pointers where the following:

- Word where all letters where capitalized
- The sequence "!!!"
- The sequence "?!" or "!"
- The sequence "..."

Other papers like [4] and [5] use more sarcasm pointers with higher complexity than what's used in this project, but with regards to time scope of the project it was decided that these four pointers where enough.

C. Logistic Regression

This model is the simplest, and should have a hard time classifying sarcasm. Because of the fact that the comments can be of different lengths and to train a model input dimension must be constant, it was decided to take all the words in the comments and take the mean of this vector to create a sentence vector for every datapoint. This meant that the model only can learn roughly what the comment was about and all relationships between words get's lost. This vector is then extended with the sarcasm pointers and is then feed through the linear layer to a single value which get's classified with a sigmoid function and a threshold. A full flow chart of the model can be found in figure 1.

D. BiLSTM with multi-head attention

This model is highly inspired by [5] with the difference that less sarcasm pointers where used. As in the Logistic Regression we first use the FastText to get the words into vectors, but instead of creating a mean sentence vector, we create a word matrix where every word is a column vector. This is then feed to the BiLSTM where the "Bi"

means that we both run the LSTM from the first to the last word and from the last to the first word, which creates two different hidden states in each time step. This so that words both before and after a specific word should be able to impact the hidden state. Now we have a matri

IV. RESULTS AND DISCUSSION

!

V. CONCLUSIONS AND OUTLOOK

VI. CONTRIBUTIONS

VII. CONFLICT OF INTEREST

VIII. DATA AND CODE AVAILABILITY

-
- [1] E. Grave, P. Bojanowski, P. Gupta, A. Joulin, and T. Mikolov, Learning word vectors for 157 languages, *Proceedings of the International Conference on Language Resources and Evaluation (LREC 2018)*, (2018), reference for the pretrained FastText model.
 - [2] D. Q. Nguyen, T. Vu, and A. T. Nguyen, BERTweet: A pre-trained language model for English Tweets, *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, , 9 (2020), for the pretrained BERTweet model.
 - [3] M. Khodak, N. Saunshi, and K. Vodrahalli, A large self-annotated corpus for sarcasm (2017), dataset that was used.
 - [4] M. S. Razali, A. A. Halin, L. Ye, S. Doraisamy, and N. M. Norowi, Sarcasm detection using deep learning with contextual features, *IEEE Access* **9**, 68609 (2021), uses the FastText, instead of the more common Word2Vec, where words get broken up into smaller bits, which handles spelling errors better , which is a common problem with social media texts.
 - [5] A. Kumar, V. Narapareddy, V. Aditya Srikanth, A. Malapati, and L. Neti, Sarcasm detection using multi-head attention based bidirectional lstm doi.org/10.1109/ACCESS.2019.2963630 (2020), a paper on the use of LSTM with Multi-Head Attention which is used in in paper.

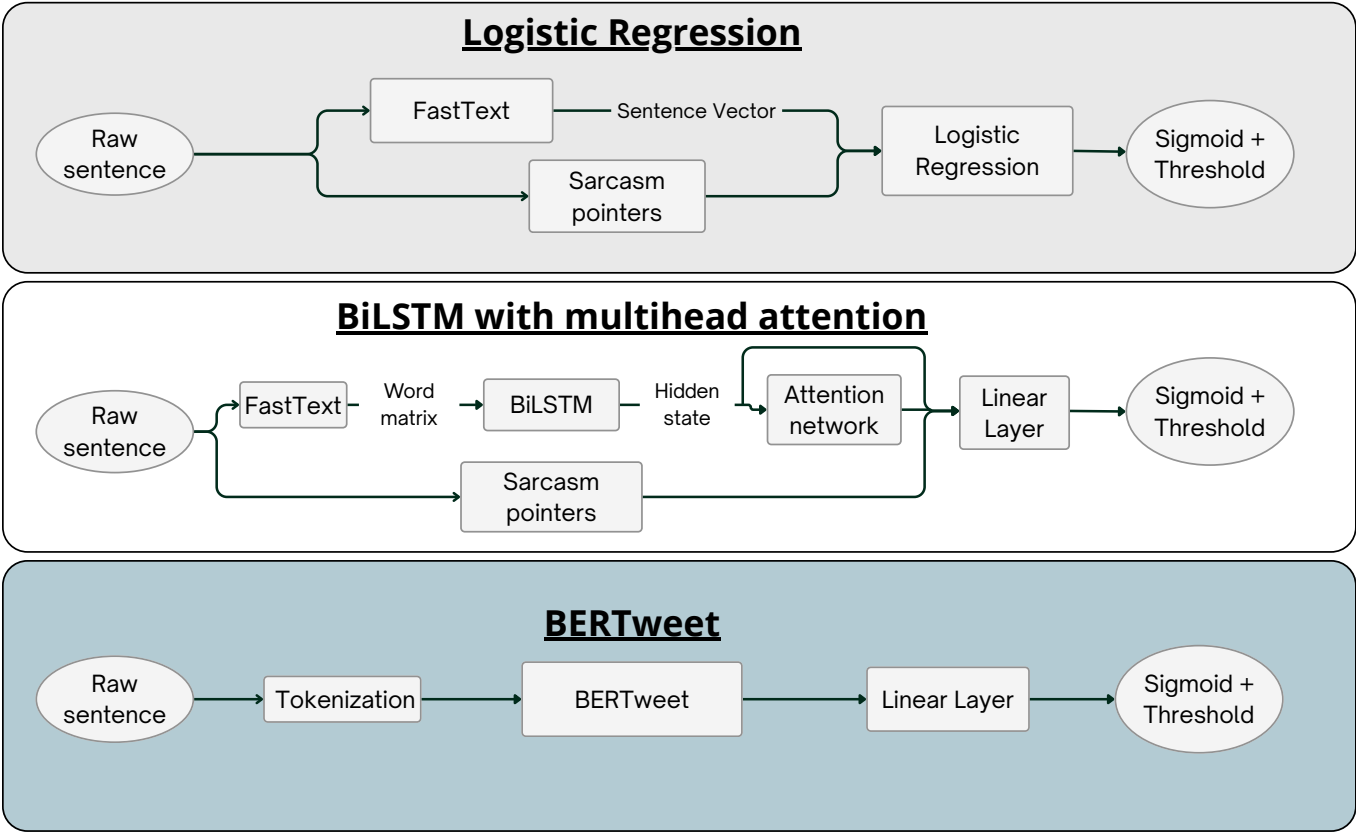


FIG. 1. **Flowchart of the models used in the project:** Architectural comparison of three sarcasm detection models. Top: a Logistic Regression baseline combining FastText sentence vectors and sarcasm pointers. Middle: a BiLSTM with multihead attention, using FastText word embeddings and sarcasm pointers as input. Bottom: a BERTweet transformer model with tokenization and a linear layer. All models output a prediction via sigmoid activation and thresholding.

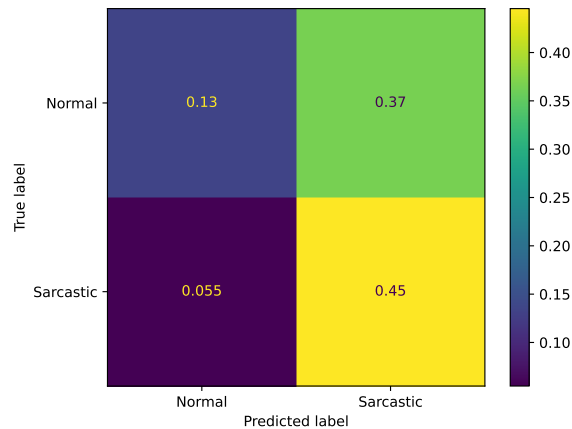


FIG. 2. **Title Figure.** Write some explanatory text for the figure. Make sure that the figure is referenced in the text. The caption should be such that, together with the figure, allows the reader to understand the concepts in the figure itself.

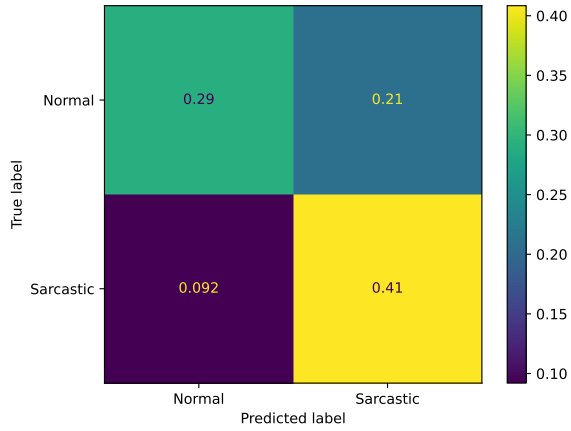


FIG. 3. **Title Figure.** Write some explanatory text for the figure. Make sure that the figure is referenced in the text. The caption should be such that, together with the figure, allows the reader to understand the concepts in the figure itself.

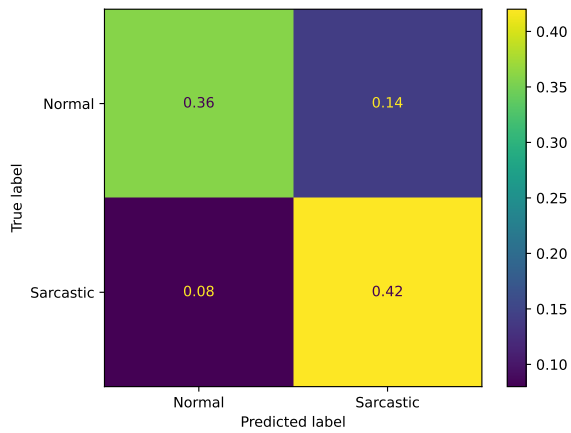


FIG. 4. **Title Figure.** Write some explanatory text for the figure. Make sure that the figure is referenced in the text. The caption should be such that, together with the figure, allows the reader to understand the concepts in the figure itself.